MSA-DATB09-03 · NEURAL NETWORKS & DEEP LEARNING FOUNDATIONS

Session 8 / 8

Sequences: RNN, LSTM and attention

A neuron with a memory, why it forgets, how the LSTM and attention fix it, then the synthesis of the course.

Guillaume Saint-Cirgue Albert School · MSc 1 October 2026

Guillaume Saint-Cirgue your instructor

Data Scientist and ML Engineer since 2016

2013 to 2016	Engineering degree, **ENSMM** (Besançon)
2016 to 2017	Data Scientist, **Rolls-Royce**
2017 to 2019	Lead ML Engineer, **GKN Aerospace**
2019	Launched **Machine Learnia** on YouTube: 200,000+ subscribers today
2020	Founded my company and my startup: **30+ AI projects** delivered since
2023	Head of Data, **Airbus**
Today	**ML-PRO** training program: 800 students

To go further

MY YOUTUBE CHANNEL

Machine Learnia

200,000+ subscribers. Machine learning and deep learning videos, including a neural network coded from scratch in NumPy.

youtube.com/@MachineLearnia

MY EBOOK

Apprendre le Machine Learning en une semaine

In English: Learn Machine Learning in One Week.

machinelearnia.com/apprendre-le-machine-learning-en-une-semaine

Both resources are in French. Click the cards in your copy of the course to open them.

Data that come in sequences

- **Text**: a sentence is a sequence of words. "The dog bites the man" is not "The man bites the dog".
- **Time series**: electricity load, prices, sensor signals.
- **Audio, video, DNA, user sessions…**

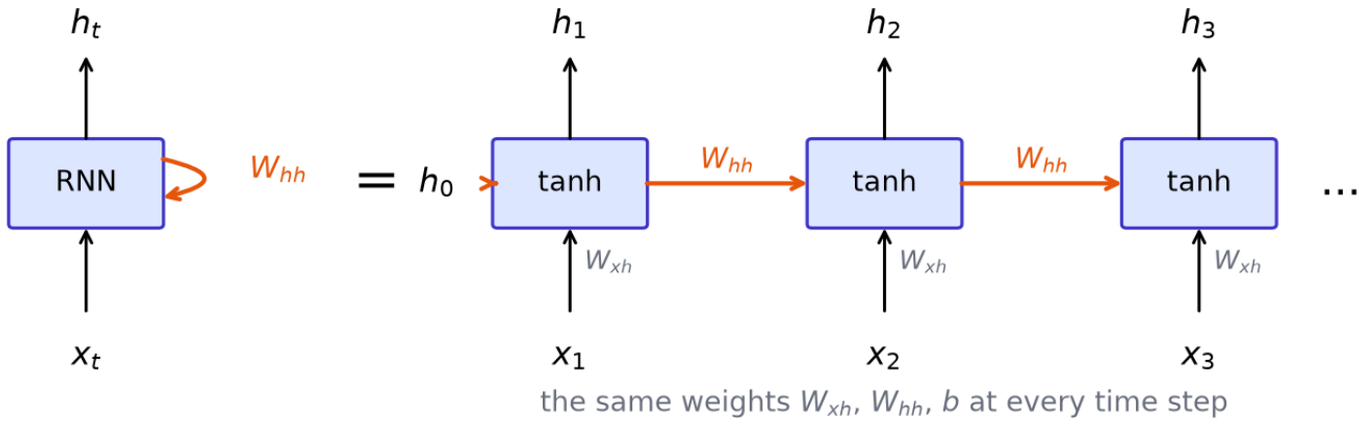
Two properties: **order matters**, and the **length varies** from one example to the next.

What our networks cannot do

- A dense network takes a fixed-size vector: one weight per position, a sentence of 12 words and one of 30 words do not fit the same layer.
- A CNN sees only a window (its receptive field) and is built for grids.
- Neither has a **memory** of what came before.

Last session: a convolution shares weights across **positions**. Today: share weights across **time steps**.

The recurrent neuron: a network with a memory



$$h_t = \tanh\left(W_{xh}\, x_t + W_{hh}\, h_{t-1} + b\right), \qquad h_0 = 0$$

- h_t (the **hidden state**, n_h numbers) summarises everything seen up to step t. Shapes: W_{xh}: (n_h, n_{in}), W_{hh}: (n_h, n_h).
- The **same** W_{xh}, W_{hh}, b at every step: the number of parameters does not depend on the length.
- For a prediction, add an output layer: $\hat{y} = W_{hy} h_T$ (one label per sequence) or $\hat{y}_t = W_{hy} h_t$ (one per step).

Exercise 1 · Three steps of an RNN by hand

A scalar RNN: $h_t = \tanh(w_x x_t + w_h h_{t-1} + b)$ with $w_x = 1$, $w_h = 0.5$, $b = 0$, $h_0 = 0$. Input sequence $x = (1, 0, -1)$.

Exercise 8 min pen, paper and calculator

1. Compute h_1, h_2, h_3 (keep 4 decimals).
2. $x_2 = 0$, yet $h_2 \neq 0$. Why?
3. With $\tanh'(z) = 1 - \tanh^2(z)$, compute $\partial h_3 / \partial x_3$ and $\partial h_3 / \partial x_1$. Which input influences h_3 the most?

Your notes / answers (saved in this browser)

The RNN in PyTorch

```
rnn = nn.RNN(input_size=8, hidden_size=16, batch_first=True)
x = torch.randn(32, 20, 8)          # 32 sequences, 20 time steps, 8 features
out, h_n = rnn(x)
print(out.shape)                    # torch.Size([32, 20, 16])   all hidden states h_1 ... h_20
print(h_n.shape)                    # torch.Size([1, 32, 16])    the last one (1 layer)
print(rnn.weight_ih_l0.shape, rnn.weight_hh_l0.shape)   # torch.Size([16, 8]) torch.Size([16, 16])
print(sum(p.numel() for p in rnn.parameters()))         # 416 = 16*8 + 16*16 + 2*16 (two bias vectors)
```

- `batch_first=True`: inputs of shape `(batch, time, features)`, like our images were `(batch, channels, H, W)`.
- Classify a whole sequence ("many-to-one"): apply a linear layer to `out[:, -1]`, the last hidden state.
- Predict at every step ("many-to-many", e.g. forecasting): apply it to every `out[:, t]`.

Backpropagation through time

Unrolled, an RNN is a deep network with one layer per time step and shared weights. Backpropagation goes back through time with the chain rule:

$$\frac{\partial h_T}{\partial h_1} = \prod_{t=2}^{T} \frac{\partial h_t}{\partial h_{t-1}} = \prod_{t=2}^{T} \operatorname{diag}\big(\tanh'(z_t)\big)\, W_{hh}$$

Scalar case: a product of $T-1$ factors $w_h \tanh'(z_t)$, with $0 < \tanh' \leq 1$.

- $|w_h| < 1$ (or saturated $\tanh$): the product goes to 0. **Vanishing**: the far past receives no gradient, the network cannot learn long dependencies.
- $|w_h| > 1$ with active neurons: the product can blow up. **Exploding**: huge steps, `nan`.

Same disease as deep sigmoid networks (session 5), but now the depth is the length of the sequence.

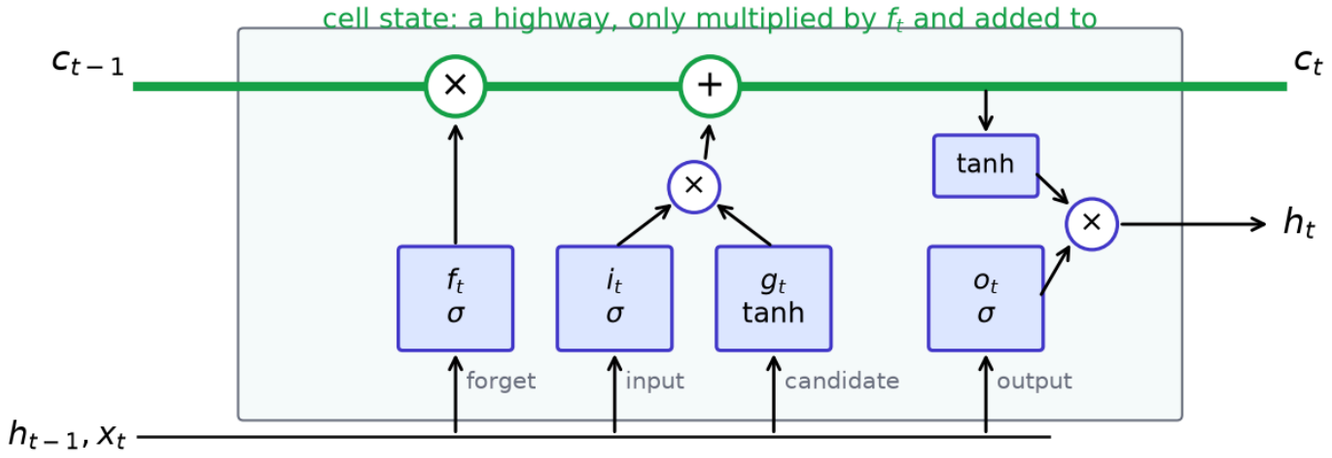
Exercise 2 · Twenty steps back

Exercise 7 min calculator

1. Ignoring $\tanh'$, the gradient that reaches 20 steps back is multiplied by w_h^{20}. Compute 0.5^{20} and 1.5^{20}. What happens to learning in each case?
2. Which remedy for the exploding case? (Session 6.)
3. In an LSTM (next slide), the gradient along the cell state is multiplied by the forget gate $f_t \in (0, 1)$ at each step. Over 30 steps, compare $f = 0.5$ (PyTorch's default initialisation, $\sigma(0)$) and $f = 0.88$ ($\sigma(2)$).

Your notes / answers (saved in this browser)

The LSTM: a cell state as a highway



Hochreiter and Schmidhuber, 1997. Gates are sigmoids: numbers in (0, 1) that let information through or not.

With $u_t = [h_{t-1}, x_t]$ (concatenation):

$$
\begin{aligned}
f_t &= \sigma(W_f u_t + b_f) && \text{forget} \\
i_t &= \sigma(W_i u_t + b_i) && \text{input} \\
g_t &= \tanh(W_g u_t + b_g) && \text{candidate} \\
o_t &= \sigma(W_o u_t + b_o) && \text{output} \\
c_t &= f_t \odot c_{t-1} + i_t \odot g_t \\
h_t &= o_t \odot \tanh(c_t)
\end{aligned}
$$

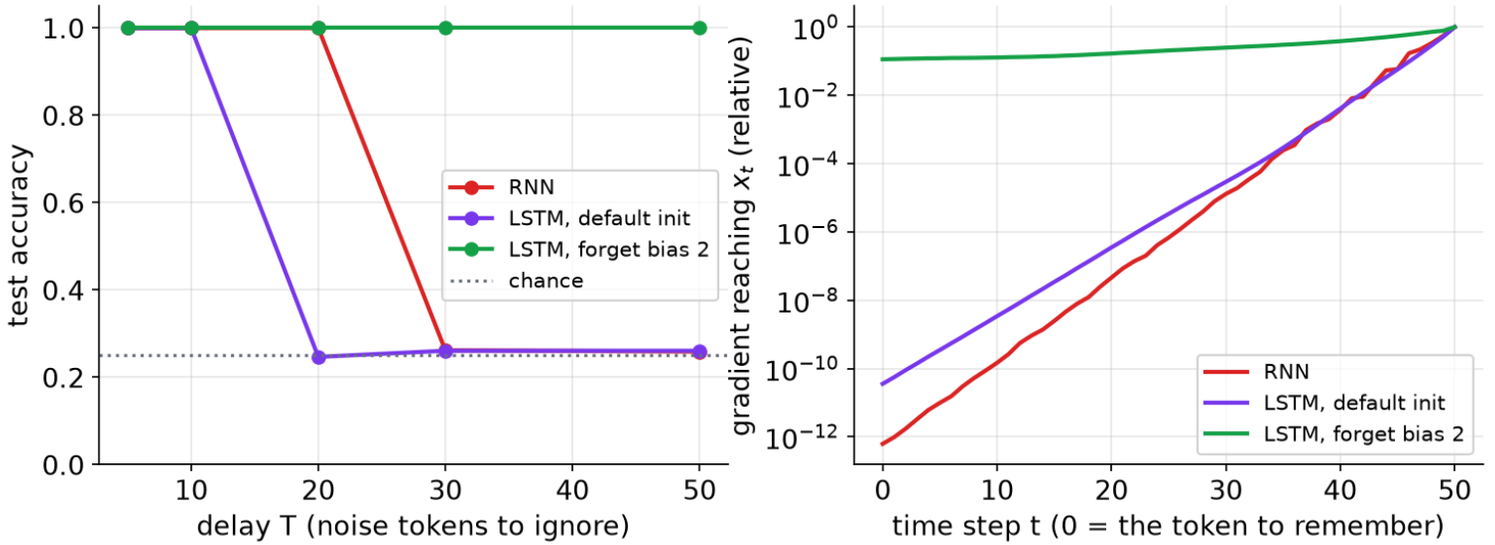
Why the LSTM remembers

Along the cell state the update is **additive**:

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t$$

$$\frac{\partial c_t}{\partial c_{t-1}} = \mathrm{diag}(f_t) \quad \text{(direct path)}$$

- No weight matrix, no $\tanh'$ on this path: only the forget gate. If the network learns $f_t \approx 1$, information and gradient cross many steps unchanged.
- Same idea as the residual connection of ResNet ($y = x + F(x)$): an identity path for the gradient.
- Initialise the forget bias to 1 or 2 so that f_t starts close to 1 (Exercise 2).



Remember the first token after T noise tokens (TP08, Part B). Left: accuracy after 400 Adam steps; right: gradient reaching each step at initialisation ($T = 50$).

GRU, and the LSTM in PyTorch

GRU (Cho et al., 2014): a lighter LSTM, no separate cell state, 2 gates.

$$z_t = \sigma(W_z u_t + b_z), \quad r_t = \sigma(W_r u_t + b_r)$$
$$\tilde{h}_t = \tanh\left(W_h [r_t \odot h_{t-1}, x_t] + b_h\right)$$
$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

Same additive trick: h_t is a mix of the old state and a candidate. 3 blocks of weights instead of 4: about 25 % fewer parameters, often similar results. (PyTorch swaps the roles of z and $1 - z$.)

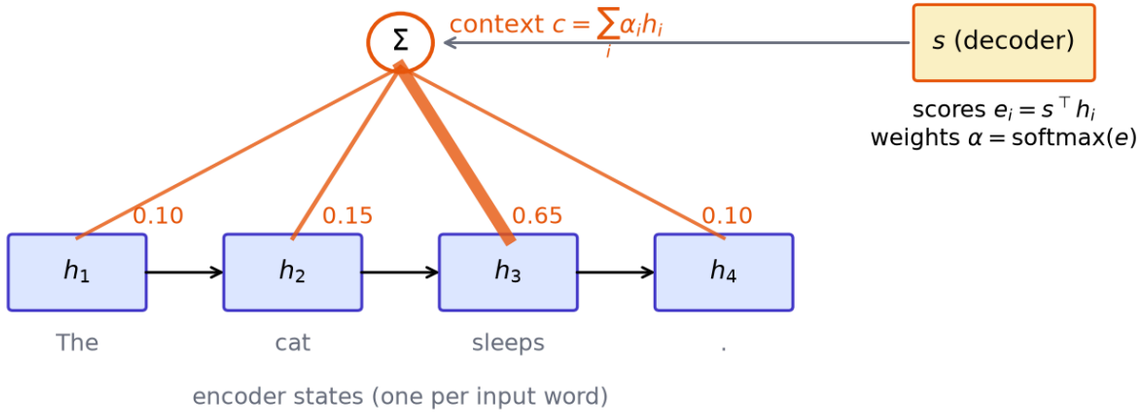
```
lstm = nn.LSTM(input_size=8, hidden_size=16,
               batch_first=True)
out, (h_n, c_n) = lstm(x)     # also returns c_n
print(out.shape)              # torch.Size([32, 20, 16])
print(c_n.shape)              # torch.Size([1, 32, 16])
print(sum(p.numel() for p in lstm.parameters()))
                              # 1664 = 4 x 416
```

RNN: 1 block, GRU: 3, LSTM: 4 (gates i, f, g, o stacked in this order).

Encoder-decoder: everything through one vector

- Translation, summarisation: a sequence in, another sequence out (Sutskever et al., 2014).
- An **encoder** RNN reads the input; its last state h_T is handed to a **decoder** RNN that writes the output word by word.
- Problem: the whole sentence must fit in one fixed-size vector h_T; quality drops for long sentences.

Soft attention (Bahdanau, Cho, Bengio, 2015): at each output step, the decoder looks again at **all** the encoder states and takes a weighted average of them, with weights it computes itself.



Soft attention: three steps

Encoder states $h_1, \ldots, h_T$, current decoder state s:

$$\underbrace{e_i = s^\top h_i}_{\text{1. scores}} \qquad \underbrace{\alpha_i = \frac{e^{e_i}}{\sum_{j=1}^{T} e^{e_j}}}_{\text{2. softmax: weights}} \qquad \underbrace{c = \sum_{i=1}^{T} \alpha_i \, h_i}_{\text{3. context vector}}$$

- A high score means "h_i is relevant to what I am producing now". The weights are positive and sum to 1.
- The context c is fed to the decoder together with s to produce the next word. A new c at every output step.
- The score can also be a small network (Bahdanau) or $s^\top W h_i$ (Luong). Everything is differentiable: trained by backpropagation with the rest.
- Bonus: the weights α show what the model looks at (next slide).

Exercise 3 · Attention by hand

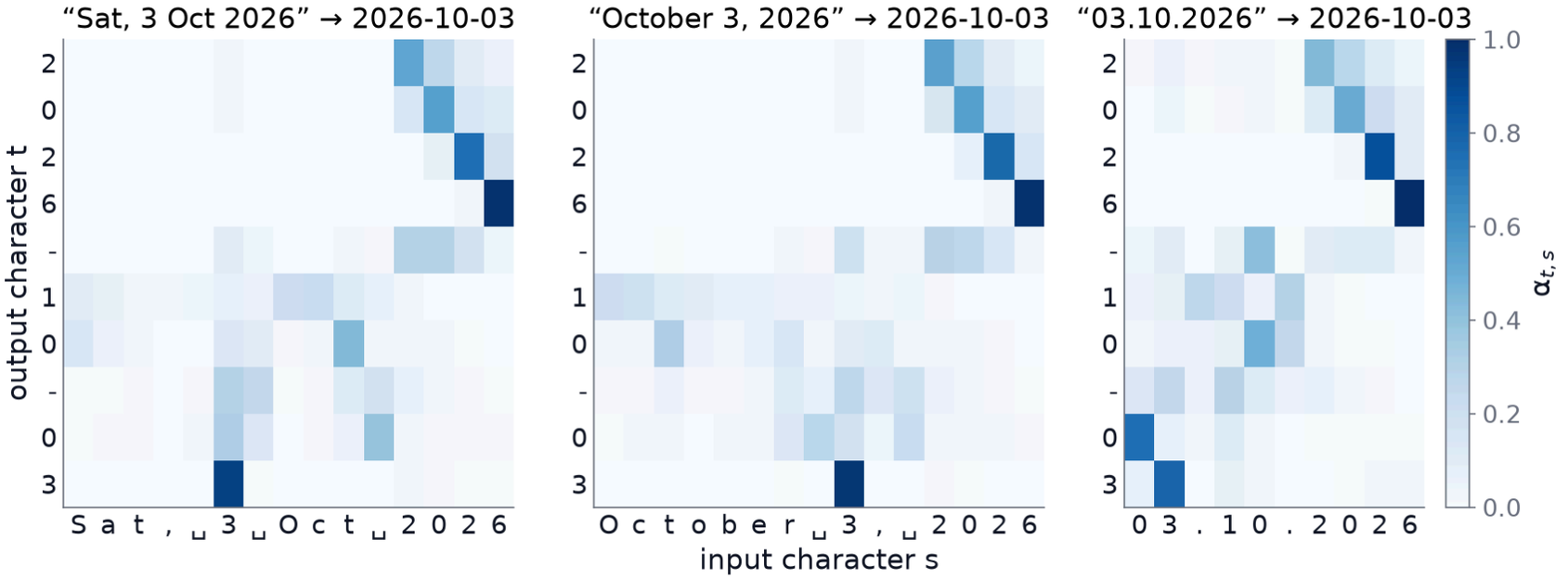
Three encoder states $h_1 = (1, 0)$, $h_2 = (0, 1)$, $h_3 = (1, 1)$, decoder state $s = (2, 1)$.

Exercise 8 min pen, paper and calculator

1. Compute the three scores $e_i = s^\top h_i$.
2. Compute the weights $\alpha = \mathrm{softmax}(e)$ (3 decimals).
3. Compute the context vector c.
4. Which encoder state does the decoder "look at"? What happens to the weights if we multiply all scores by 10?

Your notes / answers (saved in this browser)

What attention learns



Attention weights α of a small encoder-decoder RNN trained to rewrite dates in the format YYYY-MM-DD. Each output character (rows) looks at the input characters it needs (columns), whatever the input format.

Nobody told the model where the year is: the alignment is learned, only from the input and output pairs.

From attention to transformers

Self-attention (Vaswani et al., 2017, "Attention is all you need"): drop the RNN, let every position attend to every position of the same sequence.

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V$$

Queries $Q = XW_Q$, keys $K = XW_K$, values $V = XW_V$: Exercise 3 is the case of one query, with $K = V = H$.

	RNN, LSTM	Transformer
Built-in assumption	order and recency: information travels step by step	none on order (positions are added as inputs); any two positions directly connected
Long dependencies	hard (T steps of gradient)	one step
Computation	sequential in time	parallel (GPU friendly)
Data needed	moderate	a lot (hence pre-training)

Transformers, BERT, GPT: next course, MSA-DAT07-01.

Sentence embeddings: the tool of the project

```python
from sentence_transformers import SentenceTransformer, util
name = "sentence-transformers/all-MiniLM-L6-v2"
model = SentenceTransformer(name)
E = model.encode(["A cat sits on the mat.",
                  "A kitten is lying on a rug.",
                  "The stock market fell sharply today."])
print(E.shape)              # (3, 384)
print(util.cos_sim(E, E))  # cat-kitten 0.57, cat-stock 0.09
```

- Cosine similarity: $\cos(u, v) = \dfrac{u^\top v}{\|u\| \, \|v\|} \in [-1, 1]$.
- A pretrained transformer (6 layers, 22.7 million parameters, trained on a billion sentence pairs) turns any sentence into a vector of 384 numbers.
- Similar meanings, similar vectors, even without shared words ("cat" and "kitten", "mat" and "rug").
- In the project: encode the visitors' questions and the artwork descriptions, then retrieve the artwork with the highest cosine similarity.

Question-to-artwork retrieval



Cosine similarities between 5 visitor questions (rows) and 8 artwork descriptions (columns), all-MiniLM-L6-v2. Bold: the best match of each question.

- Every question finds its artwork, even with other words: "ocean, big wave" and "a huge wave", "soldiers shooting" and "firing squad".
- The similarities stay moderate (0.47 to 0.53) and an abstract question ("a painting about war and violence") can fail: measure it, and compare with a classical baseline (TF-IDF) in the project.

Break: 10 minutes

TP · RNN, LSTM and attention by hand, then sentence embeddings

TP 75 min tp/TP08_rnn_attention.ipynb

1. Part A: the RNN cell in NumPy (Exercise 1 of the course), checked against `nn.RNN`; the gradient through time with autograd (20 min).
2. Part B: RNN versus LSTM on the memory task, the forget-gate initialisation (20 min).
3. Part C: softmax, attention for one state then for a batch, scaled dot-product attention (20 min).
4. Part D: sentence embeddings, cosine similarity, question-to-artwork retrieval (15 min).

Deliverable: every cell prints "Exercise validated!"; Part D is the starting point of your project milestone.

Your notes / answers (saved in this browser)

Production training checklist (1/2): the protocol

1. **Clean splits.** Train, validation, test, decided before anything else. No leakage: normalisation statistics computed on train only, no duplicates or near-duplicates across splits (the same artist, patient or day on both sides). The test set is used once, at the end.
2. **Fixed seeds** (`torch.manual_seed`, NumPy, `random`) to reproduce a run. But one seed is one draw: report the mean and spread over 3 seeds before claiming a small gain.
3. **Baseline first.** A simple model (logistic regression, TF-IDF, the pretrained head alone) gives the number to beat. Sanity checks: initial loss close to $\ln C$, the network can overfit one batch.
4. **One change at a time**, each run logged (configuration, curves, final metrics). Two simultaneous changes cannot be told apart.

Production training checklist (2/2): training at scale

- **Learning rate first**: the most important hyperparameter. Warmup during the first steps, then cosine or step decay (session 6).
- **Checkpoint the best validation model** and stop early (session 6).
- **Mixed precision**: forward and backward in 16-bit floats on a GPU. About twice as fast, half the activation memory, same accuracy (a float32 copy of the weights is kept; with float16, the loss is scaled to avoid underflow).
- **Gradient accumulation**: the GPU fits only 16 images? Add the gradients of 4 batches, then one step: same gradient as a batch of 64.

```
accum = 4                                  # effective batch = 4 x batch
gpu = device == "cuda"
dtype = torch.float16 if gpu else torch.bfloat16
scaler = torch.amp.GradScaler(enabled=gpu)
optimizer.zero_grad()
for step, (xb, yb) in enumerate(train_loader):
    with torch.autocast(device_type=device, dtype=dtype):
        loss = loss_fn(model(xb), yb) / accum   # 16 bits
    scaler.scale(loss).backward()       # gradients ADD UP
    if (step + 1) % accum == 0:         # every 4 batches:
        scaler.step(optimizer)          # one update
        scaler.update()
        optimizer.zero_grad()
```

Checked: 4 accumulated batches of 16 (loss divided by 4) give exactly the gradient of one batch of 64.

Folklore versus evidence

What you will hear	What the evidence says
"Adam is always better than SGD."	Adam is usually faster and less sensitive to the learning rate; tuned SGD with momentum can match or beat it (classic in vision). Compare both, each with its tuned learning rate.
"A bigger batch is better."	A bigger batch is faster per epoch, not better per se: the learning rate must be adapted (with warmup), and generalisation can suffer.
"Train longer, it will improve."	Only while the validation loss improves (session 6).
"My trick gives +0.3 %."	On one seed, this is often noise. Repeat over seeds, look at the spread.

Think 3 min

A classmate reports 87.4 % with a new augmentation versus 87.1 % without, one run each, measured on the test set after trying 10 variants. What do you ask?

Your notes / answers (saved in this browser)

The whole course in one slide

Session	What we built	Its limit, fixed next
1	Perceptron, learning rule by hand	XOR, 0/1 output, no gradient
2	Sigmoid neuron: log loss, gradient $(a - y)x$, NumPy then PyTorch	still one straight line
3	A network, backpropagation by hand, autograd	we need a clean pipeline
4	PyTorch in depth: tensors, modules, losses, optimisers, the training loop	deep sigmoid networks do not learn
5	ReLU and other activations, vanishing gradient, Xavier and He	big networks overfit
6	Weight decay, dropout, early stopping, batch norm, momentum, Adam, diagnosis	dense layers ignore the structure of images
7	Convolution, pooling, receptive field, ResNet, transfer learning	fixed-size inputs, no notion of order
8	RNN, LSTM, attention, sentence embeddings, production checklist	sequential and short memory: transformers (next course)

One thread: a model, a loss, its gradient, a better architecture when the previous one hits a wall. Every fix (ReLU, He, batch norm, ResNet, LSTM) is about letting the gradient flow.

The exam

- **90 minutes, no AI**.
- **Reading and writing code** (PyTorch, as in the TPs).
- **Knowledge**: the formulas and ideas of the 8 sessions.
- **Thinking questions**: explain, diagnose, choose, justify.

How to prepare

- Redo the exercises by hand of every session without looking at the solution: a forward pass, a gradient, an update, a convolution, an output shape, a receptive field, attention weights.
- Re-run the TPs and explain every line to someone else.
- For each fix of the table above, be able to say which problem it solves and why.

Watch it on YouTube Machine Learnia, Deep Learning series

Review the foundations of this course: the whole series.

Video 1 **Introduction**
Introduction to the Deep Learning series

Video 2 **Le perceptron et la régression logistique**
The perceptron and logistic regression

Video 3 **Les gradients calculés en détail**
The gradients, computed in detail

Video 4 **Un rappel sur la vectorisation**
A reminder on vectorisation

Video 5 **Développer le tout avec NumPy**
Coding it all in NumPy

Video 6 **Application : chiens et chats**
Application: cats versus dogs

Video 7 **Réseau de neurones à 2 couches**
A neural network with 2 layers

Video 8 **La backpropagation en détail**
Backpropagation in detail

Video 9 **Programmer le réseau à 2 couches avec NumPy**
Coding the 2-layer network in NumPy

Video 10 **Réseau de neurones à N couches**
A neural network with N layers

Playlist **The whole Deep Learning series**
10 videos, from the perceptron to the network with N layers

Videos in French. Click a line in your copy of the course to open the video.

Summary

1. An RNN applies the same weights at every step, $h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b)$; its gradient through time is a product that vanishes or explodes (clipping for the latter).
2. The LSTM cell state $c_t = f_t \odot c_{t-1} + i_t \odot g_t$ is an additive highway: with f_t close to 1 the memory and the gradient survive. The GRU does the same with 2 gates.
3. Soft attention: scores, softmax, weighted sum of the encoder states. Self-attention, $\mathrm{softmax}(QK^\top/\sqrt{d})V$, is the core of the transformer.
4. Sentence embeddings + cosine similarity: retrieve the artwork closest to a question.
5. Production discipline: clean splits, seeds, baseline, one change at a time, schedules, checkpoints; mixed precision and accumulation buy speed and memory.

Project: question-to-artwork similarity with sentence embeddings, the neural versus classical comparison, and a rehearsal of your defence (tag `m4`).

Next: the exam, then MSA-DAT07-01 (deep learning for sequences and NLP: transformers).